Automated FDM 3D Print Defect Detection via Transfer Learning on CNNs

APS360: Applied Fundamentals of Deep Learning

Project Proposal | Summer 2026

Frank Hao | Frankie.hao@mail.utoronto.ca | University of Toronto

June 4, 2026

Introduction

Fused Deposition Modeling (FDM) 3D printing is now ubiquitous in engineering prototyping, yet print failures remain common and costly. Defect modes such as stringing, bed-adhesion warping, layer shifting, and under-/over-extrusion can silently ruin a multi-hour print, wasting material and time. Current practice requires a human operator to check periodically—a bottleneck that is impractical when printers run unattended overnight.

This project trains a **multi-class CNN-based classifier** to distinguish six print states—*normal*, *stringing*, *warping*, *layer shift*, *under-extrusion*, and *over-extrusion*—from webcam images captured mid-print, enabling real-time, automated quality control that can pause the printer when a defect is detected.

Deep learning is the right tool for this task. Defect appearance varies enormously across printer models, filament colors, and geometries, making rule-based classifiers brittle and unreliable. CNNs can instead learn rich visual features directly from labeled images [1]. Furthermore, transfer learning from ImageNet-pretrained models compensates for moderate dataset sizes in visual inspection settings [2], making the approach feasible without industrial-scale annotation.

Model success will be measured by per-class F_1 score and top-1 accuracy on a held-out test set, with a secondary goal of real-time inference ($\geq$10 fps) on edge hardware co-located with the printer.

Illustration

Figure 1 illustrates the end-to-end inference pipeline of the proposed system, from raw webcam image to defect classification.

Figure 1: End-to-end pipeline. A webcam frame is preprocessed and passed through a fine-tuned EfficientNet-B0 backbone; the classification head outputs one of six defect or non-defect labels. Augmentation is applied to training images only.

Background & Related Work

Obico (formerly The Spaghetti Detective) [7] is the leading open-source ML-based 3D print failure monitor. It uses a neural network to flag failures in a binary classification over webcam streams and has released a labeled image dataset. This project extends the idea to fine-grained, six-class defect categorization.

He et al. [3] introduced deep residual networks (ResNets), which allow very deep CNNs to be trained without gradient degradation. ResNets remain a standard backbone for visual classification and are evaluated as a comparison architecture in this work.

Tan and Le [4] proposed EfficientNet, achieving state-of-the-art accuracy with fewer parameters than ResNets of comparable performance through compound coefficient scaling of width, depth, and resolution. EfficientNet-B0 is the primary backbone here for its accuracy–efficiency trade-off, which is critical for edge inference.

Scime and Beuth [5] applied CNN-based anomaly detection to laser powder-bed fusion additive manufacturing, demonstrating that deep learning can identify subtle, layer-wise defects from image data alone. This validates the DL approach for AM quality control more broadly.

Pan and Yang [2] survey transfer learning, establishing that ImageNet-pretrained models fine-tuned on a target domain consistently outperform training from scratch on small or medium-sized datasets—directly motivating the pretrained EfficientNet strategy adopted here.

Data Processing

Three data sources will be combined into a unified dataset:

- **Obico Open Dataset** [7]: the primary source, containing ∼15 000 labeled printer images spanning normal prints and several failure modes.
- **Kaggle "3D Printing Failures" dataset**: a supplementary set of ∼3 000 images providing additional class coverage and printer variety.
- **Self-collected images**: ∼500 images captured from a personal FDM printer under varied lighting and filament colors, to improve domain robustness and add diversity across printer types.

All images will be resized to 224×224 pixels and channel-normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). Because normal-print frames are over-represented, weighted random sampling will ensure each class is presented at equal frequency during training. Augmentations applied only to the training split include random horizontal flip, rotation up to $\pm 15°$, and color jitter (brightness/contrast ± 0.2). The dataset is split **70% / 15% / 15%** into training, validation, and test sets, stratified by class. All transforms are implemented in `torchvision.transforms.v2` and fully reproducible from the repository.

Architecture

The proposed model uses an **EfficientNet-B0** backbone pretrained on ImageNet, with the default classifier head replaced by a Dropout layer ($p = 0.5$) followed by a linear layer outputting 6 logits. Cross-entropy loss is used with label smoothing ($\varepsilon = 0.1$) to reduce overconfidence.

Fine-tuning proceeds in two stages: (1) freeze all backbone weights and train only the classification head for 10 epochs at $\text{lr} = 1 \times 10^{-3}$; (2) unfreeze the top two MBConv blocks and fine-tune end-to-end for 20 epochs at $\text{lr} = 1 \times 10^{-4}$ with cosine annealing. The Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$) is used throughout. All training is implemented in **PyTorch** with automatic mixed precision for GPU efficiency.

Baseline Model

The baseline is a **Histogram of Oriented Gradients (HOG) + Support Vector Machine (SVM)** classifier [6]. Images are converted to greyscale; HOG features are extracted with cell size 8×8, block size 2×2, and 9 orientation bins. The resulting feature vector is fed into an RBF-kernel SVM implemented in scikit-learn. This classical pipeline requires no GPU, is fully reproducible, and represents the performance ceiling achievable without learned feature extraction. Comparison against this baseline directly quantifies the benefit of deep transfer learning.

Ethical Considerations

Intellectual property. A webcam monitoring a print bed may inadvertently capture proprietary part geometries. Any model deployment that logs webcam images must handle user data responsibly, with clear disclosure of what is recorded and stored.

Dataset and distributional bias. Training data is biased toward specific printer brands, lighting conditions, and filament types. The model may perform poorly in unseen environments, making it unreliable if deployed without validation on the target setup. All accuracy claims should be accompanied by the test conditions under which they were measured.

Automation risk. A false positive (classifying a successful print as failed) wastes material and erodes user trust. A false negative misses a real defect, wasting even more material. Deployment should enforce a confidence threshold below which the system defers to human review rather than acting autonomously.

GitHub Repository

All code, data-processing scripts, training notebooks, and model checkpoints will be maintained at the following public repository:

`https://github.com/frankhao4-max/aps360-3dprint-defect`

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010.

[3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105–6114.

[5] L. Scime and J. Beuth, “Anomaly detection and classification in a laser powder bed fusion additive manufacturing process using a trained computer vision algorithm,” *Additive Manufacturing*, vol. 19, pp. 187–196, 2018.

[6] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE CVPR*, 2005, pp. 886–893.

[7] Obico Team, “Obico: Open-source 3D printing failure detection,” *GitHub Repository*, 2023. `https://github.com/TheSpaghettiDetective/obico-server`